

操作系统原理

第十一章：文件系统

洪明坚

重庆大学软件学院

February 19, 2019

- 1 File system interface
 - File concept
 - Access methods
 - Directory structure
 - File system mounting
 - File sharing
 - Protection
- 2 File system implementation
 - File system structure
 - File system implementation
 - Directory implementation
 - Allocation methods
 - Free-space management

Outline

1 File system interface

- File concept
- Access methods
- Directory structure
- File system mounting
- File sharing
- Protection

2 File system implementation

- File system structure
- File system implementation
- Directory implementation
- Allocation methods
- Free-space management

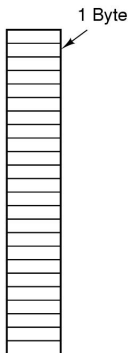
File concept

- Logical contiguous secondary storage
 - The smallest allotment of logical continuous secondary storage from a user's perspective
- Types
 - Data
 - numeric
 - character
 - binary
 -
 - Program

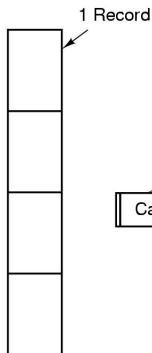
File structure (1/2)

- None - just a sequence of bytes
- Simple record structure
 - Lines
 - Fixed length
 - Variable length
- Complex structures
 - Formatted document
 - Object file
 - Executable file
- Who decides?
 - Operating system
 - Structure of the executable file and shared library file
 - Program

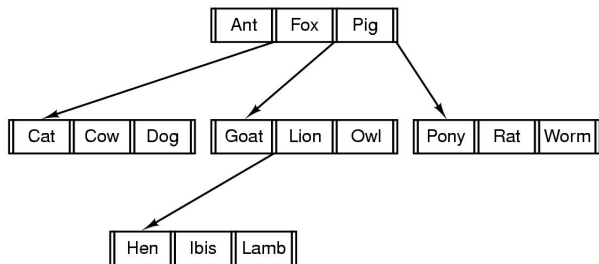
File structure (2/2)



(a)



(b)



(c)

File attributes

- Name - only information kept in human-readable form
- Type - needed for systems that support different types
- Location - pointer to data blocks on the device
- Size - current file size
- Protection - controls who can do reading, writing and executing
- Time, date and user identification - data for protection, security and usage monitoring
- All these information about files are kept in the directory structure, which is maintained on the device

File operations

- Create, Open, Close, Read, Write, Seek, Delete
- Open files
 - When a file has been opened, in addition to the information stored on the device, several pieces of data are needed to manage open files
 - File pointer - pointer to last read/write location, per process that has the file open
 - File-open count - counter of number of times a file is open to allow removal of data from open-file table when the last process closes it
 - Device location of the file - cache of data access information
 - Access rights - per-process access mode information

File types

- The content of a file determines its type.
- How does the user know the type of a file?
 - Include the type as part of the file name called the *extension*.

file type	usual extension	function
executable	exe, com, bin or none	read to run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	arc, zip, tar	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information

- Sequential access
 - read all bytes/records from the beginning
 - cannot jump around, but could rewind or back up
 - convenient when the device is magnetic tape
- Random access
 - bytes/records read in any order
 - essential for database systems

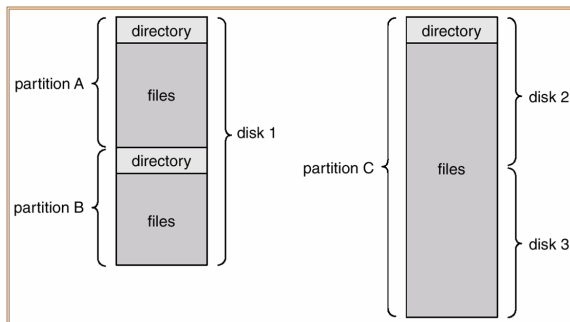
Questions

- Any questions?



Directory structure

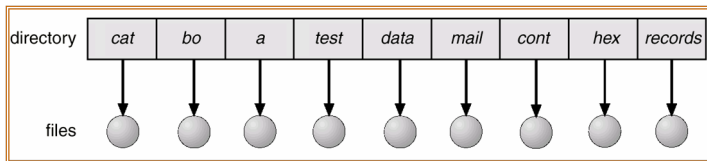
- File system resides on the devices, such as disk and tape.
 - Usually, devices are split into one or more *partitions* on which the file systems reside.
- To better organize files within a file system, *directory* is used to group files.



Directory operations

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the entire file system

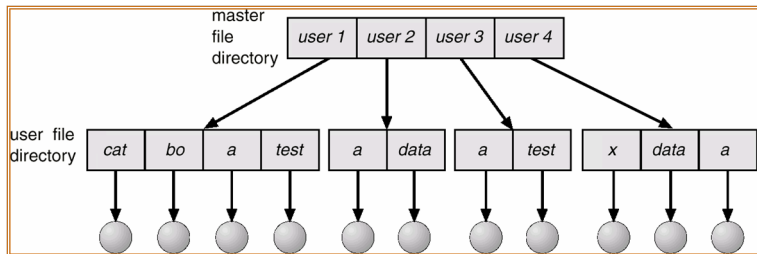
Single-level directory



- Limitations
 - Naming problem
 - Grouping problem

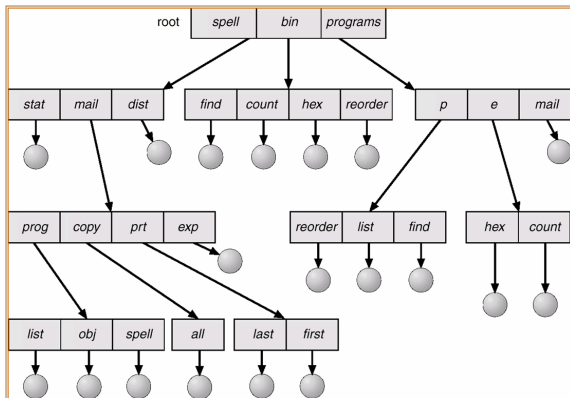
Two-level directory

- Separate directory for each user



- Features
 - Files or directories can be located by its *path*
 - Can have the same file name for different users

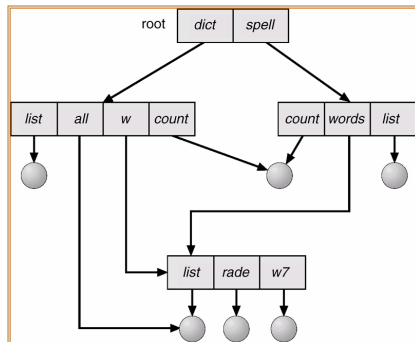
Tree-structured directory



- Features

- Grouping capability
- *Absolute* or *relative* path
- *Working (current)* directory of each process

Acyclic-graph directory



- Features

- Two different names of the same file, i.e., *aliasing*
- Dangling pointer problem
 - Some operating systems do not support acyclic-graph directory, such as MS-DOS
 - UNIX/LINUX and Windows (7+) support it via *symbol link*

Questions

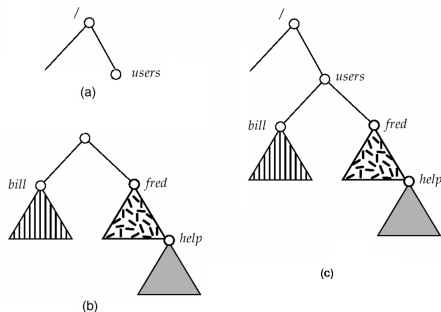
- Any questions?



File system mounting (1/2)

- A file system which resides on the device must be *mounted* to be accessed.
 - The place where the file system is mounted is named *mount point*.
 - Typically, a mount point is an empty directory.
- For example, Windows operating systems mount any partitions which contain the FAT(-12, -16, -32) or NTFS file system at “C:”, “D:” and so on during the booting process.
- While in the UNIX/Linux, system administrators must issue a command to mount a file system within a device.
 - `mount -t iso9660 /mnt/cdrom /dev/cdrom`

File system mounting (2/2)



- Example
 - An existing mounted file system, figure (a)
 - An unmounted file system resides on a disk, figure (b)
 - After mounting the file system (b) on the directory of existing file system `“/users”`, figure (c)

Questions

- Any questions?



- Sharing of files on multi-user systems is desirable.
 - Most systems identify the users by its unique *user identification*, or *UID*
 - In addition to UID, some systems also implement the *group* functionality
 - Each group is assigned an unique *group identification*, or *GID*
 - Every users can be in one or more groups
- When a file or directory is initially created, it is associated with the UID and GID of the user.
 - The user who owns a file is the *owner* of the file.

- File owner should be able to control
 - What can be done and
 - by whom
- Types of access
 - Read (R), Write (W), Execute (X), Append, Delete, List

UNIX/Linux file and directory protection

- Three classes of users and their access rights
 - **owner** with her/his RWX
 - **group** with their RWX
 - **public** with their RWX
- Example
 - `-rw-rw-r-- 1 hmj-devel525 2007-04-27 23:52 vmmap.c`

Access-control list

- The *access-control list (ACL)* specifies the user name and the types of access allowed for each user.
 - It's used to enforce fine-grained file and directory protection.
- The main problem with ACL is their length.
 - So, the most common approach is to combine the UNIX-style protection with the ACL.
 - For example, Windows NT or later and Solaris 2.6 or later use this combined approach.

Questions

- Any questions?



Outline

1 File system interface

- File concept
- Access methods
- Directory structure
- File system mounting
- File sharing
- Protection

2 File system implementation

- File system structure
- File system implementation
- Directory implementation
- Allocation methods
- Free-space management

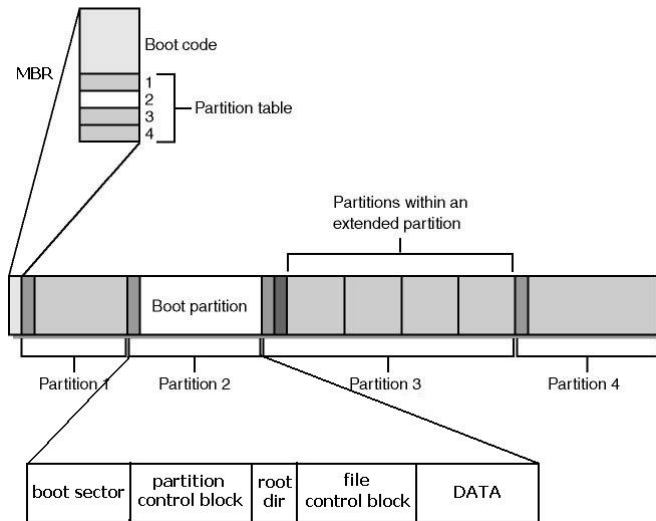
File system structure

- File
 - Logical storage unit
 - Collection of related information
 - *File Control Block (FCB)* contains all meta-information of a file
 - In UNIX, FCB is usually called *inode*
- A file system resides on the secondary storages and is organized into layers

A typical file control block

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks

On-disk file system structure (1/2)



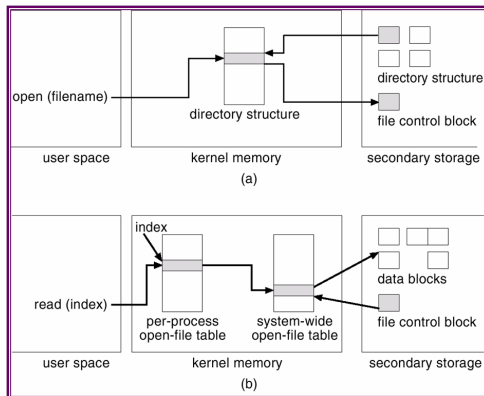
On-disk file system structure (2/2)

- The “boot sector” contains codes used to load the operating system kernel when booting;
 - It’s also known as the “boot block” and loaded by the code within the MBR.
- The “partition control block” contains the partition details such as number of blocks in the partition, size of blocks, free-block count and free-block pointers, and free FCB count and free FCB pointers.
 - It’s also known as “superblock” in UNIX or “Master File Table” in Windows NT.

In-memory file system structure

- An in-memory partition table containing information about each mounted partition
- An in-memory directory structure that holds the directory information of recently accessed directories
- The *system-wide open-file table* contains a copy of FCB of each open file, as well as other information
- The *per-process open-file table* contains a pointer to the appropriate entry in the system-wide open-file table, as well as other information

Open-file table



- The “index” in the above figure is known as *file descriptor* in UNIX or *file handle* in Windows NT.
 - It’s the value returned by the system call “open” in UNIX or “CreateFile” in Windows NT

Questions

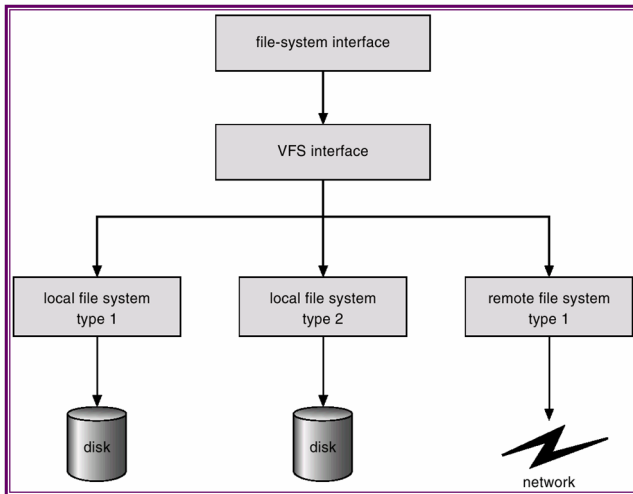
- Any questions?



Virtual File System (VFS)

- Why VFS?
 - The types of file system are as many as the flavors of ice cream.
 - How does an operating system allow multiple types of file systems to be integrated into one directory structure?
 - How can users seamlessly move between file system types as they navigate a directory?
- Most operating systems use object-oriented techniques to simplify, organize and modularize the implementation
 - A common file system interface is separated from the file system implementations
 - The file system interface contains the system calls such as “open”, “close”, “read”, “write” and “seek”, etc.

Schematic view of a VFS (1/2)



Schematic view of a VFS (2/2)

- The “VFS interface” serves two important functions:
 - It separates file-system-generic operations from their implementation by defining a clean VFS interface
 - The VFS is based on a file-representation structure, called a *vnode*, that contains a numerical designator for a network-wide unique file

Questions

- Any questions?



Directory implementation

- Some operating systems, including UNIX, treat a directory exactly as a file
 - It holds two pieces of information for each file or sub-directory within it: *file/subdir name* and *a pointer to the FCB of the file/subdir* (they are usually organized into a C “struct dirent”)
- A directory may contain a lot of files or subdirectories, how to organize these “dirent”?
 - ① Linear list
 - Simple to program
 - Time-consuming to search
 - ② Hash table
 - Decrease search time
 - Difficult to grow
- Most operating systems use the “Linear list” to organize a directory.

Questions

- Any questions?



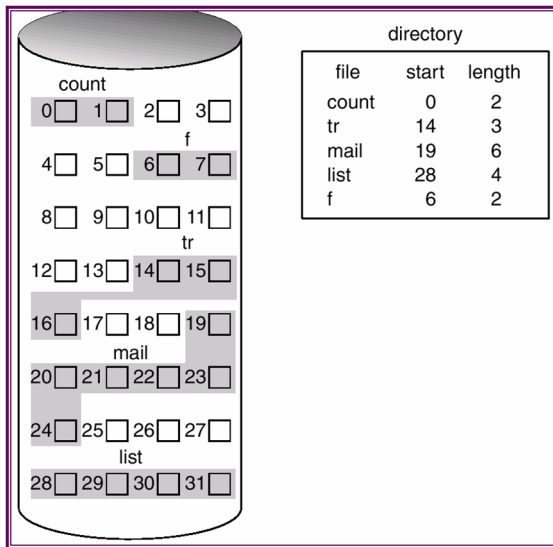
Allocation methods

- How to allocate data blocks to files so that disk space is utilized effectively and files can be accessed quickly?
- Three major methods of allocating
 - ① Contiguous allocation
 - ② Linked allocation
 - ③ Indexed allocation

Contiguous Allocation

- Each file occupies a set of contiguous blocks on the disk
- Pros and cons
 - Simple: only starting block and length (number of blocks) are saved in FCB
 - Support random access and cache friendly
 - Suffer from the *external fragmentation*
 - The problem of *dynamic storage allocation*
 - Difficult to grow a file

An example

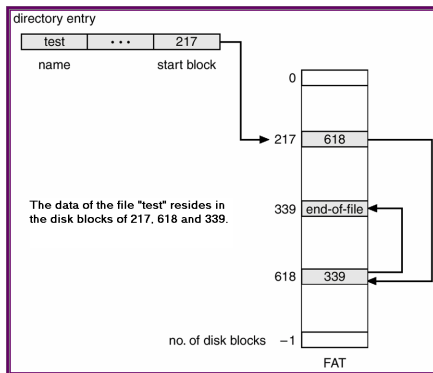


Linked Allocation

- Each file is a linked list of data blocks
 - The FCB contains a pointer to the first and last blocks of the file
 - Each block contains a pointer to the next block
 - These pointers are not visible to the user
 - Thus, if data block is 512 bytes and a block address (the pointer) requires 4 bytes, then the user sees blocks of 508 bytes
- Pros and cons
 - Simple: need only starting address
 - No waste of space
 - Extra space required for the pointer
 - No random access
 - Bad reliability because of scattered pointers
- The overhead of the pointers can be decreased by collecting several blocks into one larger block called a *cluster*

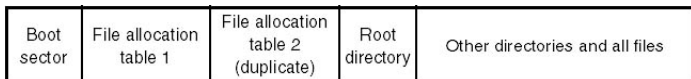
File Allocation Table (FAT)

- To solve the problem of simple linked allocation, a section of disk at the beginning of each partition is set aside to contain a table which contains all the pointers of the file system.
 - This table is called *File Allocation Table (FAT)*
- An example

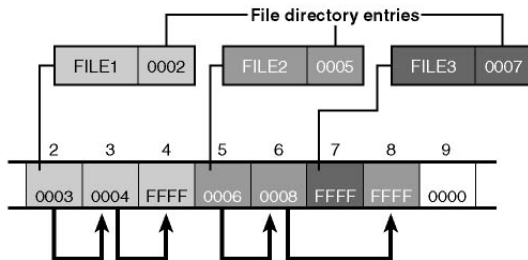


FAT file system

- The above linked allocation with FAT is used by the MS-DOS and OS/2 operating systems.
 - It's well known as the FAT file system



- An example (FAT16)



Questions

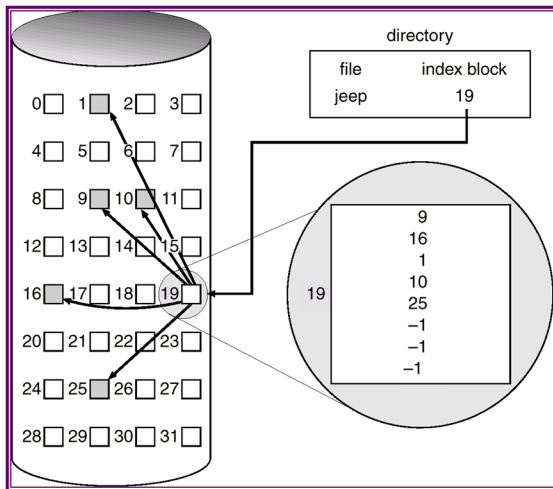
- Any questions?



Indexed Allocation

- Bring all pointers to data blocks of a file together into one location:
index block
 - The index block holds an array of data-block addresses
- Pros and cons
 - Need index table
 - Random access
 - Dynamic access without external fragmentation, but have overhead of index block.

An example of indexed allocation

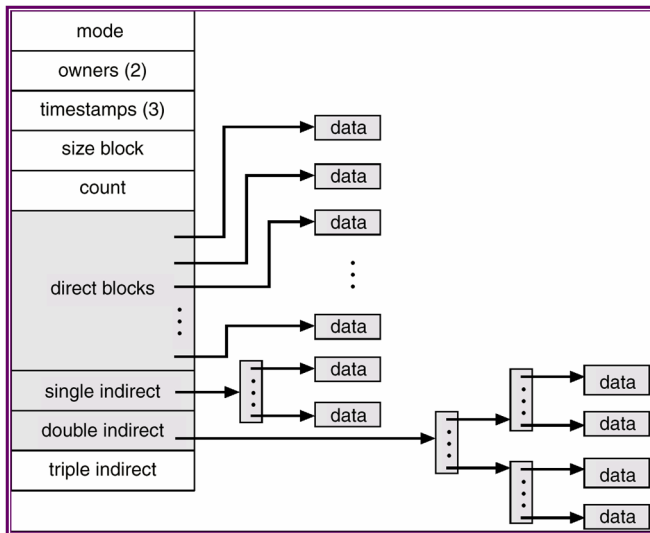


The index block

- How large should the index block be?
- Three schemes:
 - ① Linked scheme
 - ② Multi-level index
 - ③ Combined scheme of above two
 - This is scheme used by most UNIX file systems

An example

- The FCB of the UNIX file system, i.e., the *inode*



Questions

- Any questions?



Free space management

- Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
 - Hence, the free space within a file system must be recorded for allocation to new files
- Usually, one of the two methods is used
 - ① Bit vector
 - ② Linked list

Bit vector

- It's also known as *bit map*
- Each block is represented by 1 bit
 - If the block is free, the bit is 1
 - If the block is allocated, the bit is 0
- Pros and cons
 - Simple and efficient in finding the first free block, or n consecutive free blocks on the disk
 - Because many CPUs supply bit-manipulation instructions
 - Easy to get contiguous blocks
 - Bit vector must be kept in the memory to be efficient and in the disk to be persistent
 - Assume block size = 2^{12} bytes and disk size = 2^{30} bytes (1 gigabyte), then $n=2^{30}/2^{12} = 2^{18}$ (32K bytes)

Questions

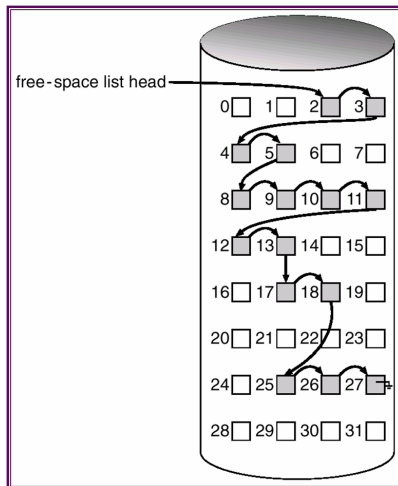
- Any questions?



Linked list

- Link together all the free disk blocks and keep a pointer to the first free block in a special location on the disk and cache it in memory for efficiency
- Pros and cons
 - The link pointer (except the link head) is saved within the free blocks, low space overhead
 - It's not efficient to traverse the list to get free blocks
 - Cache several addresses of the free blocks at beginning

An example



Questions

- Any questions?

